Cabinet: Security Self-Assessment

2026 Shenzhen FinTechathon · International Track · Topic D: Insurance Assistant

What it covers	How we keep people's insurance documents safe, what the AI is and isn't allowed to do, and the risks we know about
Live site	https://coverclarity.vercel.app (running with live AI)
Version assessed	v1.0, tagged `v1.0-submission` at https://github.com/spectacles05/cabinet, the same code that's live
Written	19 October 2026
Tests behind it	All 214 unit and integration tests and both end-to-end tests passed on 16 October 2026 (the output is saved in `submission/evidence/`). We ran the 214 again on 19 October before writing this, and they all passed (`submission/evidence/test-results-2026-10-19.txt`). No paid AI calls were made for this report.

Cabinet is a filing cabinet for your insurance. It reads your own policy documents, answers questions about them with quotes from the wording, points out gaps and double cover, and helps you put a claim together. It doesn't sell or recommend insurance, send claims or contact insurers, and there's no code in it that could do any of those things.

This report covers how we secured it, the permission levels inside the app, what we tested and what happened, and the risks that are still open. When we say something is **tested**, an automated test checks it and that test passed. If something isn't tested or isn't done, we say so. All the test data is made up: throwaway accounts that get deleted afterwards, and fictional sample policies.

---

1. How it's secured

Area	What we did
Signing in	Supabase Auth with email and password. Every page and API route checks the session with Supabase itself (`getUser()`). The redirect in `src/proxy.ts` is only there for convenience. It isn't what keeps people out.
Keeping users apart	Row-level security is switched on and forced on all 12 tables, so every row has to match `user_id = auth.uid()`. Linked rows use two-part keys `(id, user_id)`, so a row can never point at someone else's policy or document.
Server access	The app never uses the Supabase admin key (the service-role key). Every database and storage call runs as the signed-in person, so the database rules apply to our own server code too. The admin key is only used by test scripts on our own machine, and it isn't set on the live site.
Files	Two private storage buckets: policy PDFs (PDF only) and claim evidence (PDF, JPEG, PNG, WebP or HEIC, with the contents checked against the type on the server). Up to 15 MB each. Each person can only use the folder named after their own id, and files are opened through links that expire after 60 seconds.
Secrets	The Anthropic key is only read in one file, `src/lib/ai/client.ts`, which can only run on the server. Only the public Supabase address and public key reach the browser.
AI	Claude only ever sees the signed-in person's own text. It has no tools, so all it can do is send back text in a fixed format. Every quote it gives has to be found in the person's document on the page it names, or it isn't shown. Demo Mode can't create an Anthropic client at all.
Uploaded documents	We treat them as untrusted. Sentences that look like instructions to software are taken out when the PDF is read (see 3.3), and our prompts mark document text as data, not instructions.
Cost and abuse	Rate limits on each kind of request, plus daily caps on AI calls, per person and across the whole site. Over a cap, Cabinet quietly switches to the free rule-based engine.
Browser	HSTS, `X-Frame-Options: DENY`, `frame-ancestors 'none'`, `nosniff`, a strict referrer policy, a restrictive permissions policy, and `Cache-Control: no-store` on every API response.

Area	What we did
Activity log	Every workflow is logged step by step (`agent_runs` and `agent_steps`). Only 47 allowed keys can be stored, and the values have to be numbers, true/false or short ids, so no policy text, names or questions end up in it. People can add to their own log but can't edit or delete it. Judges can see their own log at `/app/runs` on the live site. It isn't in any customer menu.

These are the rate limits in the code:

What	Limit
Questions	10 a minute
Uploads	20 an hour
AI policy reads (live mode)	10 every 10 minutes
Starting and checking claims	20 a minute
Claim evidence	60 files an hour, 40 per claim
Cover Check	Once every 30 seconds
AI calls	300 per person and 1,500 across the site in any 24 hours, unless set lower (the live site is, see below)

Settings on the live site

These live in the Vercel and Supabase dashboards, not in the code, so no automated test covers them. This is how they were set on 19 October 2026.

Setting	Value	Why
`CABINET_MODE`	`live`	Everyone gets the real AI, and nobody can switch the mode
`AI_DAILY_CALLS_PER_USER`	30	Much lower than the default, so one account can't run up much cost
`AI_DAILY_CALLS_TOTAL`	200	A hard daily limit for the whole site, however many accounts are made
`SUPABASE_SERVICE_ROLE_KEY`	not set	The app doesn't need it, so we took it off
Supabase email confirmation	off	So judges can sign up and get straight in (risk 3 below)

---

2. Permission levels

Everything Cabinet does falls into one of three levels. Customers never see these. They're how we decided what the code is allowed to do.

Level	What it covers	How it's enforced	Tested?
1. Read	Reading the signed-in person's own documents, policies, notes and Claim Files, to answer questions, check cover and prepare claims. Claude only ever works at this level.	The database rules apply to every read. Search (`search_chunks`) runs with the person's own permissions and only looks at their own policies.	Tested (`rls.test.ts`, `security.test.ts`)
2. Changes the user makes	Consequential changes to policies and Claim Files are initiated by the user: saving a claim, editing policy details, ticking off a checklist step, adding evidence, deleting things. Conversations, Cover Check results and the activity log are saved automatically as part of normal use.	Each change goes through a server route that checks the session, checks the input (with zod) and writes as the person, so the database rules apply. A claim summary is checked again against the policy wording when it's saved. Deleting a policy or a Claim File asks twice, because it can't be undone.	Tested (`claims-db.test.ts`, `security.test.ts`, the end-to-end test)

Level	What it covers	How it's enforced	Tested?
3. Not allowed	Reading anyone else's data, sending claims, contacting insurers, sending messages, payments, buying or recommending products, or anything else outside Cabinet. The AI can't give itself more permissions.	There's no code for any of these. Our server code makes no outbound calls apart from the Supabase client and the Anthropic SDK, and a test scans the code for that. AI requests carry no tools. Whatever Claude sends back is matched to the person's own records, and anything that doesn't match is dropped. The mode switch is refused when the mode is fixed.	Tested (`security.test.ts`)

"Draft a message to your insurer" in Cover Check writes a message for the person to copy and send themselves. Cabinet doesn't send it.

So in short: Claude suggests text, our code checks it and decides what gets shown, and only the person's own actions change their policies and Claim Files.

---

3. What we tested

Every row here is an automated test that passed on 16 October 2026 and again on 19 October. The database tests run against the real Supabase project with throwaway accounts.

#	What we checked	Test and result
3.1	Another user sees no rows in any table, and can't download, list or upload to the owner's files, link their own rows to the owner's documents, post into the owner's conversations, change their notes or claims, or attach a claim to their policy. A signed-out visitor gets nothing.	`rls.test.ts` (10 tests) plus the cross-user tests in `claims-db.test.ts` and `security.test.ts`: all passed
3.2	None of the 5 kinds of AI request (reading a policy, answering a question, Cover Check, reading an incident, checking a claim) is sent with tools. Our server code makes no other outbound calls. Claude's output can't point at another user's policy, make up contact details or promise a payout.	`security.test.ts`, "the AI can't give itself more permissions" (3 tests): passed
3.3	Instructions planted in an uploaded policy ("ignore all previous instructions…", "Note to the AI…", `<system>` tags) are taken out when the PDF is read, while real clauses like "if you disregard the instructions of your doctor…" are kept. Even a model that "obeys" planted text can't get an answer shown, because the quote it would need is gone.	`security.test.ts`, 3 offline tests and 1 real upload: passed. Every removal is logged as `instructions_removed`.
3.4	Claim Files: evidence is stored privately and checked (a file pretending to be a photo is refused and nothing is kept), only its owner can open it, and it's removed when the Claim File is deleted. Edits are kept. Missing answers stop the policy check. Uncertain cover is called uncertain. A tampered summary loses made-up wording and contacts when it's saved.	`claims-db.test.ts` (12 tests) and `claims.test.ts` (19 tests): passed
3.5	The owner can't change their activity log: they can't delete runs or steps, change the times, reopen a finished run, or add one that's already finished or back-dated	`security.test.ts`: passed
3.6	Rate limits: the 11th question in a minute is refused with `429 rate_limited`	`security.test.ts`: passed
3.7	Cost control: over the daily cap, a question in live mode is answered by the rule-based engine with no AI request at all, and that's logged. With the backup turned off, it's refused (`429 ai_limit`).	`security.test.ts`: passed (the stand-in AI was never called)
3.8	Uploads: a non-PDF is refused (415), an oversized file is refused (413), and both are refused before anything is stored. Nobody can upload outside their own folder. Broken, scanned-image and over-long PDFs get a clear error.	`security.test.ts`, `rls.test.ts`, `pdf.test.ts` (10 tests): passed
3.9	Safe errors: unexpected errors only reach the browser as a generic message, with no database text, ids or stack traces. None of the 24 API routes sends raw error text, and every route that catches an error uses the same helper.	`security.test.ts`, "safe errors" (2 tests): passed

#	What we checked	Test and result
3.10	The log stays clean: a question with a name, address and policy number in it leaves none of them in the log. The claim journey logs each decision in order. Refusals are logged with the reason only. Deleting a policy is logged with ids and counts only.	`security.test.ts`, `claims-db.test.ts`, `redaction.test.ts`, `deletion.test.ts`: passed
3.11	When things go wrong: a failed save leaves nothing behind and is logged as failed, an AI error gives a clear message and saves nothing, and if Claude is unavailable the claim check falls back to the rule-based reader	`claims-db.test.ts` (3 tests): passed
3.12	Demo Mode never sends a request to Anthropic. Three separate checks would catch it: the client refuses to start in Demo Mode, and both the unit tests and the end-to-end test point the SDK at a fake Anthropic server that counts every request.	`demo-mode.test.ts` (26 tests), `demo-workflows.test.ts` (9 tests), the end-to-end test: passed
3.13	Browser protections: every page and API response gets the headers in section 1, API responses are never cached, and the framework doesn't advertise itself. We also checked a real production server actually sends them, not just the settings.	`headers.test.ts` (3 tests), `e2e/security-headers.spec.ts`: passed

---

4. Test results

From `npx vitest run` and `npm run test:e2e`. The full output is in evidence/test-results.txt and evidence/e2e-results.txt (16 October), and evidence/test-results-2026-10-19.txt (19 October).

Suite	Tests	16 Oct	19 Oct
Security (`security.test.ts`)	17	17 passed	17 passed
Keeping users apart (`rls.test.ts`)	10	10 passed	10 passed
Claim Files: engine, database, evidence (`claims.test.ts`, `claims-db.test.ts`)	31	31 passed	31 passed
Deletion and log redaction	4	4 passed	4 passed
Demo Mode makes no AI requests	35	35 passed	35 passed
Security headers (`headers.test.ts`)	3	3 passed	3 passed
Answers, Cover Check, policy reading, PDFs, quote checking	114	114 passed	114 passed
All unit and integration tests	**214**	**214 passed**	**214 passed**
End-to-end in a real browser: upload, ask, Cover Check, Claim File, evidence, leave and come back, edit, delete, reset, with no AI requests	1	passed	not re-run
End-to-end: security headers from a production server	1	passed	not re-run

The first end-to-end run on 16 October failed: the local test server stopped responding partway through, and we didn't capture its output, so we never found out why. We re-ran it with no code changes and both tests passed. The test server's output is now saved, so if it happens again we'll be able to see what went wrong. The database tests can also time out while setting up on a slow connection. That happened twice on 16 October, and both passed when we ran them again.

`npm audit --omit=dev` finds no vulnerabilities in anything that ships. The full `npm audit` reports 5 high-severity advisories in `braces`, which only comes in through the lint tool (`eslint-config-next`). That's only used while developing and isn't part of the live site.

5. Problems we found and fixed

What we found	The risk	What we did
People could update or delete their own `agent_runs` rows directly through the Supabase API, which would also delete the steps	The activity log could be wiped, and so could the rate limits, since they're counted from the log. That opens the door to cost abuse.	Migration `0005`: no deletes, updates limited to the status and finish time while a run is still going, and times set by the database
No daily limit on paid AI calls	One account, or a script making lots of accounts, could run up a big bill	Daily caps per person and across the site, with a quiet switch to the free engine
Instructions planted in a PDF reached the model, held back only by prompt wording and quote checks	A document written by an attacker could steer the wording, or get quoted as if it were evidence	Sentences that look like instructions are taken out when the PDF is read, before any engine or quote check sees them
Five API routes sent back raw error messages	Internal details could leak to the browser	One safe error helper on every route, and a test that checks it
Refused requests weren't logged	No record of access that was turned down	An `authorise_request` step with the reason code on every refusal
Status changes on a saved claim asked "Are you sure?"	Pointless friction on something that can be undone	Removed. Pressing the button is the permission.
The live site allowed switching between Demo and Live AI	Anyone could pick the paid mode, and the switch itself was one more thing to get wrong	`CABINET_MODE=live` on the live site, so the mode is fixed and the switch is refused (15 October)
The admin key was still set on Vercel, even though the app never uses it	If the hosting account were ever compromised, the key would bypass every database rule	Removed from Vercel (19 October)
The AI caps were at the code's defaults on the live site	1,500 AI calls a day was more than a public demo needs	Lowered to 30 per person and 200 across the site (19 October)
The browser protection headers weren't tested	A config change could quietly remove them	`headers.test.ts`, plus an end-to-end test against a real production server (16 October)
Publishing the code could have exposed personal details in the commit history	Personal details would be public	The code is published as a new repository with a single commit, with no keys or personal details in it, and we checked it from outside afterwards (19 October)

6. Known risks

Security

These are still open. None of them lets one person get at another person's data.

#	Risk	What limits it now	What we'd do before a real launch
1	**The activity log is written with the person's own login**, because we don't deploy the admin key. Someone technical could add misleading entries to their own log.	They can't change or remove existing entries (tested, 3.5), and it's only their own log	Move log writes to a separate server-only role
2	**People can edit their own rows directly through the Supabase API**, for example a saved claim summary	It only affects their own data. Cabinet checks the wording when a summary is made and when it's saved, but not again when it's shown.	Only allow direct writes to columns that don't need checking
3	**Email confirmation is off and there's no CAPTCHA**, so anyone can make lots of accounts	The site-wide cap of 200 AI calls a day, however many accounts there are	Turn on email confirmation and add a CAPTCHA

#	Risk	What limits it now	What we'd do before a real launch
4	**Rate limits are per account, not per IP address**, so someone using many accounts gets more	The site-wide AI cap	Add rate limiting by IP
5	**The prompt-injection filter is based on patterns** (8 of them), so cleverly worded instructions could get past it	The main defences don't depend on it: no tools, a fixed output format, every quote checked, and results matched to the person's own records. Anything that gets through can only ever be quoted as what it is.	Keep adding patterns as we find new tricks
6	**The Content Security Policy only covers framing, base URI, forms and objects**, not scripts	React escapes what it displays, and we never insert raw HTML	A nonce-based `script-src`
7	**Cross-site request protection relies on SameSite=Lax cookies**, and the mode and reset routes also need a JSON content type. There are no CSRF tokens.	Browsers don't send the session cookie on cross-site POSTs. This is built but not tested.	Add CSRF tokens, and a test
8	**Account deletion is by request**, and the contact address (`CONTACT_EMAIL`) isn't set, so the Contact page is hidden and there's currently no way to ask	People can delete each policy and Claim File themselves, along with its files	Self-serve account deletion, and a contact address
9	**Our integration tests use the same Supabase project as the live site**, with the admin key from a local file that isn't in the repository	Test accounts are throwaway and deleted afterwards, and the tests only use made-up data	A separate test project, and automatic test runs
10	**Claude can still misread wording that's really there**	Every point links to its quote, Cabinet says "can't confirm" when it isn't sure, it never promises a claim will be paid (tested), and the terms page says it isn't advice	Bigger benchmarks with real policies

Regulation: this needs a specialist's review before any real launch

Calling something "guidance" doesn't take it outside regulation. Our samples are UK-style, so we looked at the UK first.

- **Advice on insurance (UK, FCA).** Cabinet explains the cover people already have. Two Cover Check actions get close to advice on a particular contract: "Draft a message to your insurer" (for example, asking about adding an endorsement) and "Compare the wording" for possible double cover. To stay on the right side, Cabinet never recommends buying, cancelling or switching anything, and it sticks to facts with quotes. *Not sure:* whether a regulator would see pointing out a gap as a personal recommendation.
- **Claims help.** Helping someone prepare a claim could count as "assisting in the administration and performance of a contract of insurance" if it's done as a business, and UK claims-management rules cover some kinds of claim. Cabinet only organises the person's own information and their policy's own wording. The person decides, and sends the claim themselves. Cabinet never represents them, negotiates or contacts the insurer. *Not sure:* what permissions, if any, a commercial version would need.
- **Data protection (UK and EU GDPR).** Policies contain personal data, and claims can include **health data**, which is special category data. That needs a legal basis under Article 9 (probably explicit consent) and a data protection impact assessment. We'd also need processor agreements and transfer arrangements with Anthropic (US), Vercel and Supabase, a retention schedule, and a way to handle access and deletion requests. Our app runs in Washington DC (Vercel's `iad1`) and the database appears to be in Europe, so data already crosses borders. *Today:* sign-up asks people to confirm they understand document text is sent to Anthropic, and the privacy policy covers claims, health details and processing outside their country, and asks people not to upload real documents while it's a prototype.
- **Telling people it's AI.** If Cabinet were offered in the EU, Article 50 of the AI Act says people must be told they're dealing with an AI system. We tell people at sign-up and in the privacy policy, but the chat itself doesn't carry an AI label. *Before an EU launch:* add a notice where people ask questions. Cabinet doesn't price or underwrite anything, so the AI Act's high-risk category for insurance shouldn't apply.

- **Mainland China (where the event is).** Offering Cabinet in China would need separate consent under PIPL for sensitive personal data and for sending data abroad (with a security assessment or standard contract). NFRA rules limit internet insurance business, including claims services, to licensed institutions. And the Anthropic API may not be available there. *Not sure, and we haven't looked into it in depth.*
- **Consumer protection.** Answers mustn't mislead. Every statement about cover links to the exact wording, uncertainty is said plainly, and nothing Cabinet produces can promise a claim will be paid (tested for claims, questions and the Demo Mode answers).

---

7. Examples from a real activity log

We made these with `npm run evidence:logs` on 15 October 2026. A throwaway test account ran the main workflows against the real database in Demo Mode, its own log was exported, and then the account was deleted. The script also checks the exported file has none of the person's words, email or id in it. The full file is evidence/execution-log-sample.json (13 runs, 50 steps). We've relabelled the ids to make them easier to read.

Answering a question (`qa`, succeeded):

```
{ "step": "retrieve_user_chunks",  "metadata": { "policy_id": "wallet", "retrieved_chunk_count": 15 } }
{ "step": "prepared_demo_answer",  "metadata": { "model": "demo-prepared-v1", "app_mode": "demo",
"completed": true } }
{ "step": "verify_citations",      "metadata": { "assessment": "excluded", "citation_count": 5,
"rejected_citation_count": 0, "condition_count": 1, "reply_word_count": 52 } }
{ "step": "save_conversation",     "metadata": { "thread_id": "thread-1" } }
```

Preparing and saving a claim (three `claim` runs: understand the incident, check the cover, save once the person confirms):

```
{ "step": "understand_incident_rules", "metadata": { "incident_type": "theft", "app_mode": "demo" } }
{ "step": "choose_questions",          "metadata": { "policy_id": "policy-1", "policy_count": 3,
"question_count": 2 } }
{ "step": "verify_claim_evidence",     "metadata": { "verdict": "conditional", "fact_count": 2,
"deadline_count": 2, "contact_count": 0, "checklist_count": 3, "dropped_count": 0 } }
{ "step": "await_user_confirmation",   "metadata": { "completed": false } }
{ "step": "user_confirmed_summary",    "metadata": { "completed": true, "policy_id": "policy-1" } }
{ "step": "save_claim",                "metadata": { "claim_id": "claim-1" } }
```

A refused request (`action`, failed). Only the reason is kept:

```
{ "step": "authorise_request", "status": "failed", "metadata": { "denied_reason": "not_found",
"error_code": "not_found" } }
```

Reading an upload (`ingest`), including how many planted instructions were taken out:

```
{ "step": "parse_pdf_pages", "metadata": { "document_id": "document-1", "page_count": 6,
"instructions_removed": 0 } }
```

In live mode, AI runs also log `ai_budget_check` (`{ "completed": true, "budget": "ok" }`) and the model step with `model`, `input_tokens` and `output_tokens`. The tests in section 3 cover both.

8. Checking it yourself

```
npm install
npx vitest run         # 214 tests; the database ones need .env.local with Supabase test details
npm run test:e2e       # builds the app, clicks through it in Demo Mode (no AI requests), and checks the
security headers
npm run evidence:logs  # makes a fresh log sample like the one in section 7 (needs the Supabase test
details)
npm audit --omit=dev   # checks the packages that ship
```

The database setup is in `supabase/migrations/0001` to `0006`. `0004` adds claims, `0005` is the security hardening from section 5, and `0006` adds claim evidence.